
accelerate
collaborate
innovate
communicate
palm
PalmSource
Conference 2000



The GCC Tool Chain

John Marshall
Toolsmith

PalmSource
Conference 2000

A Little GCC History

- 1984: The GNU project, and later the Free Software Foundation and GCC are born
- July 1997: prc-tools 0.5.0 released, containing
  - GCC 2.7.2.2 (from January 1997)
  - GNU linker and debugger of similar vintage
  - Build-prc, a custom post-linker
  - Header files for Palm OS® software version 2.0
- October 1999: GCC 2.95.2 released
- February 2000: prc-tools 2.0 released
  - Then current GCC, linker, debugger, etc.
  - Interoperability with past and future Palm OS® SDKs

Why Choose GCC?

- It doesn't cost very much
- It's free
- Good error reporting and standards compliance
- The optimiser is correct
  - Very few problems due to turning optimisation on
- Speed and size of optimised code?
  - I think the jury is still out on that one

Contents of a Tool Chain for the Palm OS® Platform

- Something to compile the source code
  gcc, as, ld
- Something for resources
  pilrc or other options
- A “post-linker” to tie the two together
  build-prc
- The Palm OS SDK
  - Headers
  - Libraries
- Not strictly necessary, but very nice to have
  - Poser, a debugger, code examples, an IDE

Some Detailed Case Studies

- Applications with multiple code resources
- C++ vtables and global data
- Projects which aren't applications:
  - Shared libraries
  - HackMaster hacks
  - Standalone code resources ("plugins")

Multiple Code Section Example

```
/* foo.c */
#include “foobar.h”  …declares foo() in the main
                      section, and bar1() and
                      bar2() in “barsec”
UInt32 PilotMain (…) {
   if (…normal launch…)
      return bar1() + bar2();
   }
int foo() { return 0; }

/* bar.c */
#include “foobar.h”
int bar1() { return foo(); }
int bar2() { return 0; }
```

Specifying Code Sections in GCC

- Annotating function declarations (preferably in header files):

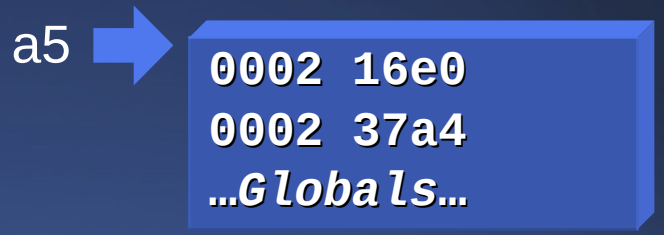
```
int bar1()
    __attribute__ ((section (“barsec”)));
```

- Specifying a section for a whole translation unit in a linker script:

```
SECTIONS { barsec { bar.o(.text) }}
```

Unfortunately, this doesn’t work on Palm OS®

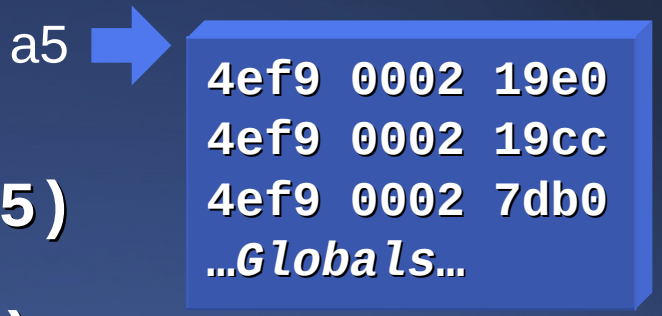
Code Generated by GCC




```
PilotMain:
   246d fffc    movea.l -4(%a5),%a2
   4eaa 0000    jsr 0(%a2)
   …
   4eaa 0010    jsr 16(%a2)

bar1:
   206d fff8    movea.l -8(%a5),%a0
   4ea8 0098    jsr 152(%a0)
```

Code Generated by CodeWarrior




```
PilotMain:
   4ead fff4    jsr -12(%a5)
   …
   4ead fffa    jsr -6(%a5)

bar1:
   4ead ffee    jsr -18(%a5)

In the global data:
   4ef9 0002 7db0    jmp 0x216ea
   4ef9 0002 19cc    jmp 0x219cc
   4ef9 0002 19e0    jmp 0x219e0
```

Features of the Two Implementations

- Overhead in the (precious!) global data:
  - GCC: 4 bytes for each code resource
  - CW: 6 bytes for each function called from a function in a different code resource
- Execution time:
  - GCC: one 2 word jump instruction, plus 4 words due to a load—which is often shared between calls
  - CW: a total of 5 words of jump instructions
- Code size for each call between code sections:
  - GCC: at most 8 bytes; perhaps 6 on average
  - CW: 4 bytes (always)

Selecting a Code Resource for a Whole Translation Unit (.o file)

- CodeWarrior supports this, GCC doesn't:
  - CW: just move files around in the Segment window
  - GCC: you can use a generic GNU linker script to arrange .o files into sections, and build-prc will dutifully emit several code resources

    But the code will crash: in the current Palm OS® GCC implementation, the compiler needs to know the exact section for each function at compile time, and rearranging things at link time is too late
- Lots of people say this is convenient

Selecting a Code Resource for Each Function Separately

- GCC supports this, CodeWarrior doesn't
- This is more flexible—these two arrangements are not necessarily the same:
  - Source files arrange functions by encapsulation and visibility
  - Code resources arrange functions by frequency of invocations and reachability requirements
- Some people say this is inconvenient
  - Inconsistent attributes lead to crashes
  - You have to write lots and lots of attributes

Ensuring Consistency of Function Declarations and Attributes

- Each function called from outside its module should have exactly one declaration
  - If you're not following this rule, your project is probably in trouble anyway
- Compiler warnings are your friend
  - -Wimplicit-function-declarations
  - -Wmissing-declarations
  - -Wredundant-decls

Writing Lots and Lots of Attributes

- Defining a macro saves typing and helps when switching attributes on and off:

```
#define BARSEC \
   __attribute__((section (“barsec”)))

int foo();
int bar1() BARSEC;
int bar2() BARSEC;
```

- You only have to do it once!

C++ vtables and Global Data

- In both CodeWarrior and GCC, the virtual function entries in the vtables are pointers to functions, *i.e.*, absolute addresses

```
move.l (object),%a2
move.l 4(%a2),%a0
jsr (%a0)
```

- Code moves, so these addresses need to be adjusted at application startup
- But code is read only

C++ vtables and Global Data

- But what if they were relative jumps instead?

```
move.l (object),%a2
jsr 4(%a2)
…
bra.w foo
```

- Simple implementation might be limited to a single code section
- But even the simple implementation isn't very simple

Specifying Generic Code Resources

- Shared libraries, Hacks, plug-ins are fundamentally all the same thing to the linker

```
code 1000 <foo>
trap {  EvtGetEvent <getevent>
        WinDrawChars <drawch>
        }
```

- How to specify the filename
  - What about filenames on the command line?
  - What directory is a filename relative to?
- Syntactic sugar allows “code” and “TRAP” resources to number themselves

ANN: prc-tools 2.1 pre 1

- As always, source code in SourceForge CVS
- Binary RPM for Linux
- Miscellaneous small fixes since 2.0:
  - Multiple code resource applications in Flash
  - Floating point libraries
  - Tools built with non-shared C++ support
  - Build-prc reads files in PRC format, understands “subdir/tFRM03e8.bin”, handles options correctly
- Experimental new option (don’t use this!):
  - -mexperimental-return-reg-d0

Contributing



Your compiler needs you!

- http://prc-tools.sourceforge.net/
- http://gcc.gnu.org/

Contributing to prc-tools

- Web site at http://prc-tools.sourceforge.net/
  - FAQs, tutorials, documentation
- Working on the tools:
  - Debugging multiple code resource applications
  - Resolving the shared library and GLib saga
    - Are globals accessed via A4 or A5?
  - The large projects covered earlier
- Bug reports


accelerate
collaborate
innovate
communicate
palm
PalmSource
Conference 2000

Last Year's TO DO List

- Submit the GCC, binutils, gdb patches back to the main FSF distributions
- Move from producing COFF files to ELF
  - Automatic stripping of unused functions
  - Slight optimisation of short jumps
  - Multiple code resources without attributes
- C++ support
  - Virtual functions accessible without globals
  - Exceptions
  - The standard library

"Simple things should be simple, and complex things should be possible."

—Alan Kay


accelerate
collaborate
innovate
communicate
palm
PalmSource
Conference 2000